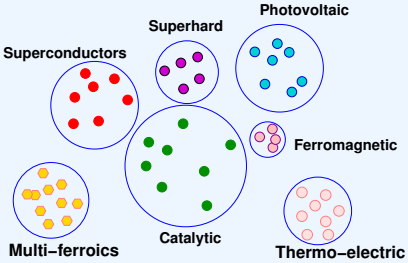
APPLICATIONS OF GRAPH LAPLACEANS: CLUSTERING

- The clustering problem; Basic method: K-means
- Similarity graphs; kNN graphs
- Measures of separation: edge cuts, normalized cuts, etc
- Application: Segmentation.

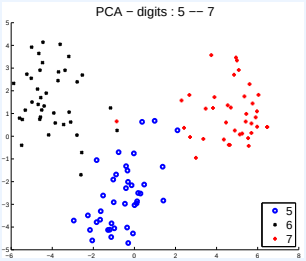
Problem:

Group n data items: $x_1, x_2, \cdots, x_n$, into k clusters so that items in each cluster are similar in some sense.

- Example: materials



- Example: Digits



- Each group is a 'cluster' or a 'class'
- 'Unsupervised learning'

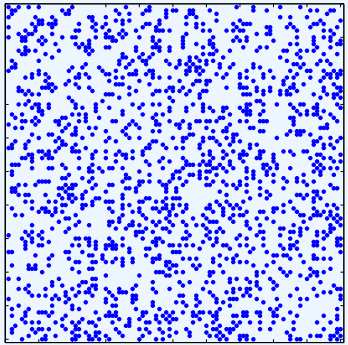
What is 'Unsupervised Learning'?

Ans: Class of methods that do not exploit labeled data

- Example of digits: perform a 2-D projection
- Images of same digit tend to cluster (more or less)
- Such 2-D representations are popular for visualization
- Can also try to find natural clusters in data, e.g., in materials
- Basic clusterning technique: K-means

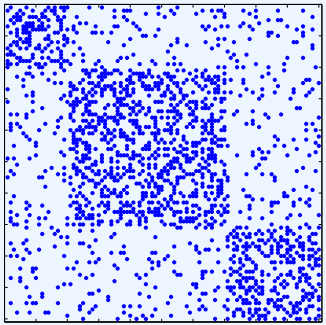
Example: Community Detection

- Communities modeled by an 'affinity' graph [e.g., 'user A sends frequent e-mails to user B'] . [data: www-personal.umich.edu/~mejn/netdata/]



← Original Adj. matrix

Goal: Find ordering so blocks are as dense as possible →



- Use 'blocking' techniques for sparse matrices
- Advantage of this viewpoint: need not know # of clusters.

Example of application Data set from :

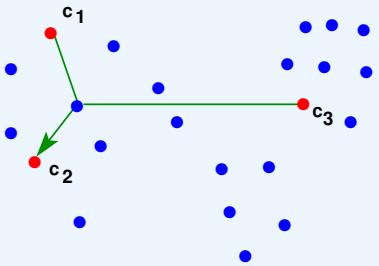
http://www-personal.umich.edu/~mejn/netdata/

- Network connecting bloggers of different political orientations [2004 US presidentual election]
- 'Communities': liberal vs. conservative
- Graph: $\mathbf{1,490}$ vertices (blogs) : first $\mathbf{758}$: liberal, rest: conservative.
- Edge: $i \rightarrow j$: a citation between blogs i and j
- Blocking algorithm (Density theshold=0.4): subgraphs [note: density = $|E|/|V|^2$.]
- Smaller subgraph: conservative blogs, larger one: liberals

A basic method for clustering: K-means

- A basic algorithm that uses Euclidean distance

1. Select p initial centers: $c_1, c_2, \ldots, c_p$ for classes $1, 2, \cdots, p$
2. For each x_i do: determine class of x_i as $\text{argmin}_k \|x_i - c_k\|$
3. Redefine each c_k to be the centroid of class k
4. Repeat until convergence



- Simple algorithm
- Works well but can be slow
- Performance depends on initialization

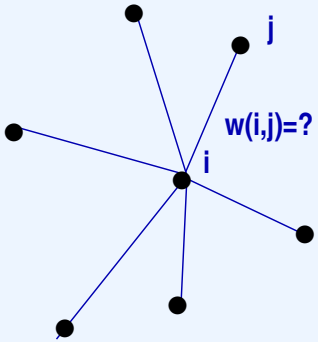
Methods based on similarity graphs

- Class of Methods that perform clustering by exploiting a graph that describes the similarities between any two items in the data.
- Need to:
  1. decide what nodes are in the neighborhood of a given node
  2. quantify their similarities - by assigning a weight to any pair of nodes.

Example: For text data: Can decide that any columns i and j with a cosine greater than 0.95 are 'similar' and assign that cosine value to w_{ij}

First task: build a 'similarity' graph

- Goal: to build a similarity graph, i.e., a graph that captures similarity between any two items



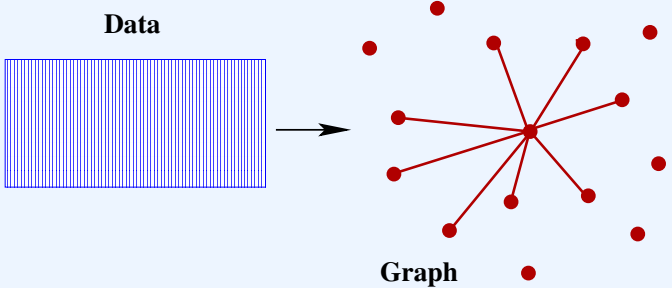
- Two methods: K-nearest Neighbor graphs or use Gaussian ('heat') kernel

K-nearest neighbor graphs

- Given: a set of n data points $X = \{x_1, \ldots, x_n\} \rightarrow$ vertices
- Given: a proximity measure between two data points x_i and x_j – as measured by a quantity $dist(x_i, x_j)$
- For each point x_i list of the k 'nearest neighbors' of x_i (edges between x_i and these nodes).
- Note: graph will usually be directed $\rightarrow$ may need to symmetrize

Nearest neighbor graphs

- For each node, get a few of the nearest neighbors → Graph



- Problem: How to build a nearest-neighbor graph from given data
- We will revisit this later.

Two types of nearest neighbor graph often used:

ϵ-graph: Edges consist of pairs (x_i, x_j) such that $\rho(x_i, x_j) \leq \epsilon$

kNN graph: Nodes adjacent to x_i are those nodes x_ℓ with the k with smallest distances $\rho(x_i, x_\ell)$.

- ϵ-graph is undirected and is geometrically motivated. Issues: 1) may result in disconnected components 2) what ϵ?
- kNN graphs are directed in general (can be trivially fixed).
- kNN graphs especially useful in practice.

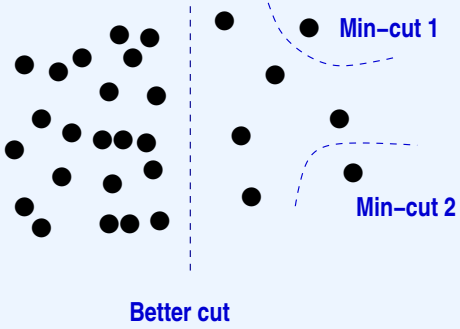
Similarity graphs: Using 'heat-kernels'

- Define weight between i and j as: $w_{ij} = f_{ij} \times \begin{cases} e^{\frac{-\|x_i - x_j\|^2}{\sigma_X^2}} & \text{if} \quad \|x_i - x_j\| < r \\ 0 & \text{if} \quad \text{not} \end{cases}$
  - Note $\|x_i - x_j\|$ could be any measure of distance...
  - f_{ij} = optional = some measure of similarity - other than distance
  - Only nearby points kept.
  - Sparsity depends on parameters

Edge cuts, ratio cuts, normalized cuts, ...

- Assume now that we have built a 'similarity graph'
- Setting is identical with that of graph partitioning.
- Need a Graph Laplacean: $L = D - W$ with $w_{ii} = 0, w_{ij} \geq 0$ and $D = diag(W * ones(n, 1))$ [in matlab notation]
- Partition vertex set V in two sets A and B with $A \cup B = V, \quad A \cap B = \emptyset$
- Define $$cut(A, B) = \sum_{u \in A, v \in B} w(u, v)$$

- First (naive) approach: use this measure to partition graph, i.e.,
  ... Find A and B that minimize $cut(A, B)$.
- Issue: Small sets, isolated nodes, big imbalances,



Ratio-cuts

- Standard Graph Partitioning approach: Find A, B by solving

  Minimize $cut(A, B)$, subject to $|A| = |B|$

- Condition $|A| = |B|$ not too meaningful in some cases, restrictive in others.
- Minimum Ratio Cut approach. Find A, B by solving: Minimize $\frac{cut(A,B)}{|A|.|B|}$
- Difficulty: find solution (original paper [Wei-Cheng '91] proposes heuristics)
- Approximate solution : spectral .

Theorem

[Hagen-Kahng, 91] If λ_2 is the 2nd smallest eigenvalue of L, then a lower bound for the cost c of the optimal ratio cut partition, is: $c \geq \frac{\lambda_2}{n}$.

Proof Consider an optimal partition A, B and let $p = |A|/n, q = |B|/n$. Note that $p + q = 1$. Let x be the vector with coordinates $x_i = \begin{cases} q & \text{if } i \in A \\ -p & \text{if } i \in B \end{cases}$. Note that $x \perp \mathbb{1}$. Also if (i,j) == an edge-cut then $|x_i - x_j| = |q - (-p)| = |q + p| = 1$, otherwise $x_i - x_j = 0$.

Therefore: $x^T L x = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2 = w(A,B)$. In addition:

$\|x\|^2 = pq^2n + qp^2n = pq(p+q)n = pqn = \frac{|A|.|B|}{n}$. Therefore, by the Courant-Fischer theorem: $\lambda_2 \leq \frac{(Lx,x)}{(x,x)} = n \times \frac{w(A,B)}{|A|.|B|} = n \times c$. Hence result. ■

✦ Idea: use eigenvector associated with λ_2 to determine partition, e.g., based on sign of entries. Ratio-cut measure will determine where to split.

Normalized cuts [Shi-Malik,2000]

- Recall notation $w(X,Y) = \sum_{x \in X, y \in Y} w(x,y)$ - then define:

$$\text{ncut}(A,B) = \frac{cut(A,B)}{w(A,V)} + \frac{cut(A,B)}{w(B,V)}$$

- Goal is to avoid small sets A, B

✍ 1 What is $w(A,V)$ in the case when $w_{ij} == 1$?

- Let x be an indicator vector:

$$x_i = \begin{cases} 1 & if \quad i \in A \\ 0 & if \quad i \in B \end{cases}$$

- Recall that: $x^T L x = \sum_{(i,j) \in E} w_{ij} |x_i - x_j|^2$ (note: each edge counted once)

- Therefore:

$$cut(A,B) = \sum_{x_i=1, x_j=0} w_{ij} = x^T L x$$

$$w(A,V) = \sum_{x_i=1} d_i = x^T W \mathbb{1} = x^T D \mathbb{1}$$

$$w(B,V) = \sum_{x_j=0} d_j = (\mathbb{1} - x)^T W \mathbb{1} = (\mathbb{1} - x)^T D \mathbb{1}$$

- Goal now: to minimize ncut

$$\min_{A,B} \text{ncut}(A,B) = \min_{x_i \in \{0,1\}} \frac{x^T L x}{x^T D x} + \frac{x^T L x}{(\mathbb{1} - x)^T D x}$$

- Let $k = \dfrac{\sum_{x_i>0} d_i}{\sum_i d_i}$; $\quad b = \dfrac{k}{1-k}$; and: $\quad y = (\mathbb{1} + x) - b(\mathbb{1} - x)$

- Then it can be shown that we need to solve:

$$\min_{y_i \in \{1,-b\}} \frac{y^T L y}{y^T D y}$$
$$\text{Subject to} \quad y^T D\, \mathbb{1} = 0$$

- + Relax $\rightarrow$ need to solve Generalized eigenvalue problem

$$Ly = \lambda Dy$$

- $y_1 = \mathbb{1}$ is eigenvector associated with eigenvalue $\lambda_1 = 0$
- y_2 associated with second eigenvalue solves problem.

A few properties

✍ 2 Show that

$$ncut(A,B) = \sigma \times \frac{cut(A,B)}{w(A,V) \times w(B,V)}$$

where σ is a constant

✍ 3 How do ratio-cuts and normalized cuts compare when the graph is d-regular (same degree for each node).

Extension to more than 2 clusters

- Just like graph partitioning we can:

1. Apply the method recursively [Repeat clustering on the resulted parts]
2. or compute a few eigenvectors and run K-means clustering on these eigenvectors to get the clustering.

Application: Image segmentation

- First task: obtain a graph from pixels.
- Common idea: use “Heat kernels”
- Let F_j = feature value (e.g., brightness), and Let X_j = spatial position.

✦ Then define:

$$w_{ij} = e^{\frac{-\|F_i - F_j\|^2}{\sigma_I^2}} \times \begin{cases} e^{\frac{-\|X_i - X_j\|^2}{\sigma_X^2}} & \text{if} \|X_i - X_j\| < r \\ 0 & \text{else} \end{cases}$$

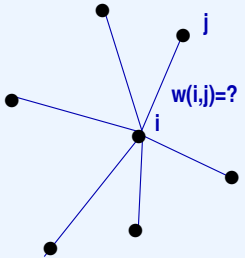
✦ Sparsity depends on parameters

✍ 4 Run *test_mir* and *test_jump*

Spectral clustering: General approach

1 Given: Collection of data samples $\{x_1, x_2, \cdots, x_n\}$

2 Build a similarity graph between items



3 Compute (smallest) eigenvector (s) of resulting graph Laplacean

4 Use k-means on eigenvector (s) of Laplacean

✦ For Normalized cuts solve generalized eigen problem.